PREMIER UNIVERSITY

Department of Computer Science & Engineering

Complex Engineering Assignment

Course Name : Big Data Analytics

Course Code : CSE 4345 – (V4)

Semester : 8th

Section : "B"

Batch : 42nd

Session : Spring 2026

Submitted By:

Name	ID
Md Imtiaz Uddin	0222220005101132
Sabiha Taslim Jannath	0222220005101134
Pritam Nath	0222220005101142
Nusrat Jahan Ananna	0222220005101099

Submitted To:

Rokan Uddin Faruqui
Professor
Department of Computer Science & Engineering
University of Chittagong, Chittagong

Date of Submission: 2nd November, 2026

Remarks:

EpiScope: A Big Data Analytics Platform for Disease Surveillance, Outbreak Monitoring, and Resource Planning in Bangladesh

Md Imtiaz Uddin, Sabiha Taslim Jannath, Pritam Nath, and Nusrat Jahan Ananna
Department of Computer Science and Engineering
Premier University, Chittagong, Bangladesh
Course: CSE 4345 – Big Data Analytics

***Abstract*—EpiScope is a Big Data Analytics project designed to demonstrate an integrated approach to disease surveillance, outbreak monitoring, and healthcare resource planning in Bangladesh.**

The platform follows a Bronze–Silver–Gold data architecture to ingest, validate, transform, and aggregate epidemiological and supporting datasets. The system processes encounter records, facility information, population statistics, weather data, and geographical references to generate weekly district-level disease summaries and facility-reporting information.

The analytics component implements baseline outbreak detection, next-week disease forecasting, resource demand estimation, and demand-prioritized resource allocation. An interactive visualization dashboard developed using D3.js provides disease and district filters, key performance indicators, weekly disease trends, district comparisons, and geographic map integration.

The project was completed through the coordinated contributions of four members responsible for data engineering, analytics, visualization, and testing and integration. In a documented integration run, 100 encounter records resulted in 98 accepted records and two quarantined records. Additionally, all 3,840 population records passed validation. The latest recorded full automated test run passed 38 tests with zero failures in 58.79 seconds. However, several requirements, including complete privacy suppression, role-based access, dashboard usability validation, formal scalability experiments, and advanced analytical evaluation, require further verification.

EpiScope therefore provides an educational engineering foundation for data-driven disease surveillance while documenting its current capabilities and limitations.

***Index Terms*—Big Data Analytics, Disease Surveillance, Apache Spark, Data Engineering, Outbreak Detection, Forecasting, Resource Allocation, D3.js, Data Privacy.**

I. Introduction

A. Background

Public health surveillance is an important activity for identifying disease patterns, monitoring outbreaks, and supporting healthcare resource planning.

Healthcare information is often collected from multiple facilities and administrative regions. These datasets may contain different structures, formats, reporting periods, and levels of completeness.

Without a structured data-processing system, it becomes difficult to combine information, identify unusual disease patterns, and generate reliable analytical summaries.

EpiScope was developed as a Big Data Analytics project to demonstrate how heterogeneous epidemiological datasets can be processed and analyzed through a structured data engineering and analytics platform.

The project focuses on Bangladesh and uses district-level information to demonstrate disease surveillance and resource planning workflows.

EpiScope is an academic prototype rather than a clinically validated public-health system. Synthetic or prepared project data must not be interpreted as actual disease incidence.

B. Problem Statement

The main problem addressed by EpiScope is the transformation of heterogeneous epidemiological data into reliable, traceable, and useful analytical information.

The system needs to handle:

- Different data sources and file formats.
- Missing, invalid, and duplicate records.
- Patient privacy and sensitive information.
- District-level disease aggregation.
- Delayed epidemiological reporting.
- Unusual disease case increases.
- Short-term disease forecasting.
- Healthcare resource demand and allocation.
- Interactive visualization of analytical results.

C. Project Objectives

The major objectives are:

1) Develop a structured data ingestion pipeline.
2) Implement Bronze, Silver, and Gold processing layers.
3) Validate data quality and quarantine invalid records.
4) Apply privacy-aware transformations.
5) Generate weekly district-disease summaries.
6) Implement outbreak detection and forecasting.
7) Estimate healthcare resource demand.
8) Develop resource allocation functionality.
9) Create an interactive visualization dashboard.
10) Verify system components using automated tests.

II. Stakeholders and Requirements

A. Stakeholders

The intended stakeholder groups include:

- National-level public-health decision-makers.
- District-level health officers.
- Healthcare facilities.
- Data analysts and researchers.
- Data and platform administrators.

National users require broad comparisons across districts, while district-level users require locally relevant disease information.

Analysts require reliable datasets and data lineage. Administrators require monitoring, validation, and secure data management.

B. Engineering Conflicts

The project involves several important engineering trade-offs.

III. Complex Engineering Problem Attributes

The EpiScope project demonstrates several complex engineering characteristics.

IV. System Architecture and Design

A. Overall Architecture

EpiScope follows a medallion-style data architecture consisting of three primary processing layers.

1) Bronze Layer
2) Silver Layer
3) Gold Layer

These layers prepare data for downstream analytics and visualization.

B. Technology Stack

The project uses the following technologies:

TABLE III
Technology Stack

Technology	Purpose
Python	Data processing and analytics
Apache Spark	Distributed data processing
PySpark	Python interface for Spark
Spark SQL	Analytical queries
Parquet	Structured data storage
JSON	Configuration and data exchange
pytest	Automated testing
HTML/CSS	Dashboard interface
JavaScript	Dashboard functionality
D3.js v7	Data visualization
Git/GitHub	Version control
PowerShell	Development environment

C. Bronze Layer

The Bronze layer is responsible for ingesting source datasets while preserving their original information and adding metadata.

The metadata includes:

- Source information.
- Ingestion timestamp.
- Batch identifier.
- SHA-256 file checksum.
- Additional provenance information.

Tabular datasets are stored in Parquet format. Geographic GeoJSON files are handled separately.

D. Silver Layer

The Silver layer validates and standardizes the ingested data. It performs:

- Required-field validation.
- Date validation.
- Facility and district verification.
- Duplicate detection.
- ICD-10 code validation.
- Patient identifier hashing.
- Age-band conversion.
- Quarantine of invalid records.

E. Gold Layer

The Gold layer generates analytical datasets from validated Silver records.

The main outputs are:

- Weekly district-disease summaries.
- Weekly facility-reporting summaries.
- Data quality reports.
- Source batch lineage.

Gold datasets are stored in Parquet and partitioned by epidemiological year and week.

The principal Spark SQL tables are:

- `episcope.weekly_district_disease`
- `episcope.weekly_facility_reporting`
- `episcope.quality_report`

F. Incremental Processing

A persistent JSON ledger records completed batches and their processing information.

The orchestration process checks whether a batch has already completed. Completed batches can be skipped unless forced execution is requested.

The system also identifies affected epidemiological weeks and recomputes selected Gold partitions using available Silver batches.

The documented incremental test verifies that:

- An affected week's case count is updated.
- Lineage includes the contributing batches.
- An unaffected week's count remains unchanged.

V. Data Quality, Lineage, and Privacy

A. Data Validation

The Silver encounter pipeline checks the following conditions:

1) Missing encounter ID.
2) Missing patient ID.
3) Missing facility ID.
4) Missing district ID.

TABLE I
MAJOR ENGINEERING CONFLICTS

Conflict	Description	Project Approach
Early Detection vs False Alerts	Sensitive detection thresholds may identify unusual increases but can generate false alerts.	A rolling historical baseline is used for initial outbreak detection. Further calibration is required.
Spatial Detail vs Privacy	District-level information is useful but small counts may expose individuals.	Patient identifiers are hashed and exact age is reduced to age bands. Full suppression remains to be verified.
Resource Efficiency vs Equity	Prioritizing high-demand districts may disadvantage other districts.	A demand-prioritized allocation approach is implemented. Equity constraints remain future work.
Freshness vs Computing Cost	Frequent processing improves data freshness but increases computational cost.	Batch and incremental processing are implemented.
Complexity vs Explainability	Advanced analytical methods may be difficult to explain.	Transparent baseline methods are used for initial analytics.

TABLE II
MAPPING OF P1–P7 ATTRIBUTES

Attribute	Project Context	Evidence
P1 – Depth of Knowledge	Combining big data processing, epidemiological analytics, privacy, and visualization.	PySpark, Spark SQL, Parquet, and D3.js.
P2 – Conflicting Requirements	Balancing privacy, analytical detail, detection sensitivity, and resource allocation.	Hashing, age bands, and baseline detection.
P3 – Depth of Analysis	Validating records and evaluating analytical outputs.	Automated tests and integration results.
P4 – Unfamiliar Issues	Handling inconsistent schemas, missing reference data, and late-arriving records.	Schema normalization, temporary test fixtures, and incremental recomputation.
P5 – Beyond Standard Codes	Combining public-health requirements with engineering decisions.	Data lineage, aggregation, and interactive visualization.
P6 – Multiple Stakeholders	Supporting national and district-level information needs.	Disease and district filters.
P7 – Interdependence	Pipeline, analytics, serving exports, dashboard, and testing depend on one another.	Four coordinated development responsibilities.

5) Unknown district.
6) Invalid onset date.
7) Invalid report date.
8) Report date earlier than onset date.
9) Reporting delay greater than 21 days.
10) Invalid age.
11) Missing sex.
12) Missing ICD-10 code.
13) Invalid ICD-10 code.
14) Unknown facility.
15) Duplicate encounter ID.

Invalid records are separated into quarantine outputs with validation reason codes.

B. Encounter Processing Results

TABLE IV
ENCOUNTER INTEGRATION RESULTS

Metric	Result
Input Records	100
Accepted Records	98
Quarantined Records	2
Pass Rate	98%

The two quarantined records were associated with a duplicate encounter ID and an invalid age.

C. Population Processing Results

TABLE V
POPULATION INTEGRATION RESULTS

Metric	Result
Input Records	3,840
Accepted Records	3,840
Quarantined Records	0
Pass Rate	100%
Districts	64
Age Groups	20
Sex Categories	3

D. Patient Privacy

The Silver pipeline uses salted SHA-256 hashing to replace patient identifiers.

The secret salt is obtained from the external environment variable:

```
PATIENT_ID_SALT
```

The implementation rejects a missing salt or a salt shorter than 32 characters.

Exact age is converted into age bands. Raw patient identifiers and exact age are excluded from the cleaned Silver encounter output.

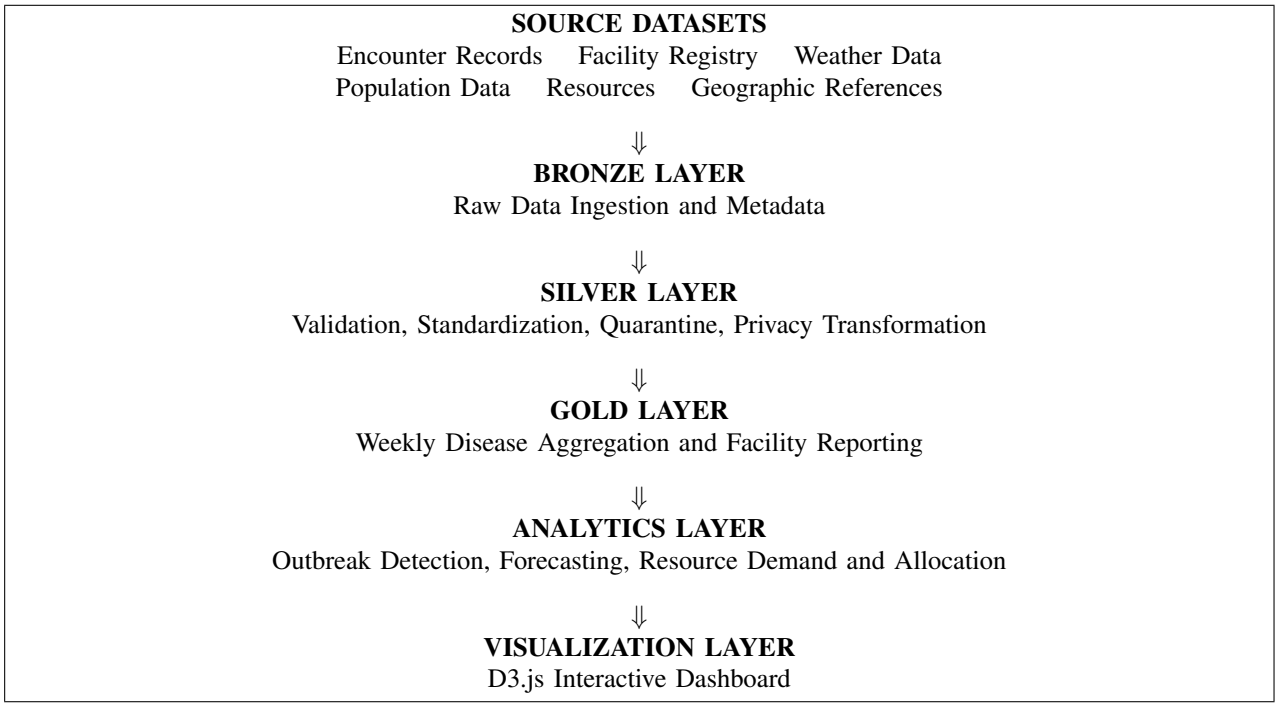


Fig. 1. Overall EpiScope System Architecture

Hashing is pseudonymization and does not guarantee complete anonymity.

The assignment also requires small-cell suppression, protection against subtraction recovery, role-based access, and privacy analysis. The supplied reports do not establish that all these controls have been implemented and tested. They must be treated as pending verification.

E. Data Lineage

Gold aggregation records contributing source batch identifiers through the `source_batch_ids` field.

This supports traceability by allowing analysts to identify which batches contributed to an aggregated result.

VI. Analytics Methods

The analytics component is implemented in:

`analytics/analytics.py`

A. Outbreak Detection

The `detect_outbreaks()` function groups records by district and disease.

It calculates a historical baseline using up to the previous four weekly observations, requiring at least two prior observations.

A potential outbreak is flagged when the current case count is positive and at least twice the available baseline, subject to the configured threshold.

This provides a simple and explainable baseline method.

Further calibration and evaluation using historical or generated ground truth remain necessary.

B. Disease Forecasting

The `forecast_next_week()` function estimates next-week disease cases.

It uses up to the latest four available weekly observations and calculates their average.

The forecast is rounded to produce the estimated number of cases.

The method is simple and understandable, but its accuracy must be evaluated using appropriate forecasting metrics before making predictive-performance claims.

C. Resource Demand Estimation

The `estimate_resource_demand()` function estimates resource requirements based on forecasted cases.

The calculation is:

$$\text{Resource Demand} = \text{Forecasted Cases} \times \text{Resource Factor}$$

For example:

$$20 \times 0.2 = 4$$

Therefore, 20 forecasted cases require an estimated 4 resource units when the resource factor is 0.2.

The function rounds demand up to a whole resource unit.

D. Resource Allocation

The `optimize_allocation()` function distributes limited resources among districts.

It:

- Aggregates demand by district.

- Sorts districts by estimated demand.
- Prioritizes districts with higher demand.
- Allocates available resources.
- Calculates unmet demand.
- Ensures allocation does not exceed available supply.

The current implementation uses a demand-prioritized greedy algorithm rather than a full mathematical optimization model.

Equity constraints and formal optimization remain future improvements.

VII. Dashboard and Visualization

A. Dashboard Overview

The visualization component was developed using HTML, CSS, JavaScript, and D3.js v7.

The main dashboard files include:

```
dashboard/index.html
dashboard/js/dashboard.js
dashboard/js/d3.v7.min.js
```

The dashboard includes:

- Disease filter.
- District filter.
- Reset control.
- KPI summary cards.
- Weekly disease trend chart.
- Cases-by-district chart.
- Bangladesh district map integration.

B. Dashboard Data Sources

The documented dashboard data inputs include:

```
dashboard/data/
    weekly_district_disease.json
    district_centroids.csv
    bangladesh_adm2.geojson
```

The JSON file provides aggregated disease and district values. The CSV supports district-name and identifier matching. GeoJSON provides geographic boundaries.

C. Dashboard Interactions

The disease filter allows users to select a disease or view all diseases.

The district filter allows users to select a district or view all districts.

The reset control restores the default selections.

KPI cards display total reported cases, districts represented, and disease categories for the current view.

The weekly trend chart presents reported cases across epidemiological weeks. The district chart compares cases across districts.

The map code supports district selection to update the district filter. The dashboard is a static HTML/CSS/JavaScript application and should be served locally over HTTP rather than opened directly from disk, because browser restrictions

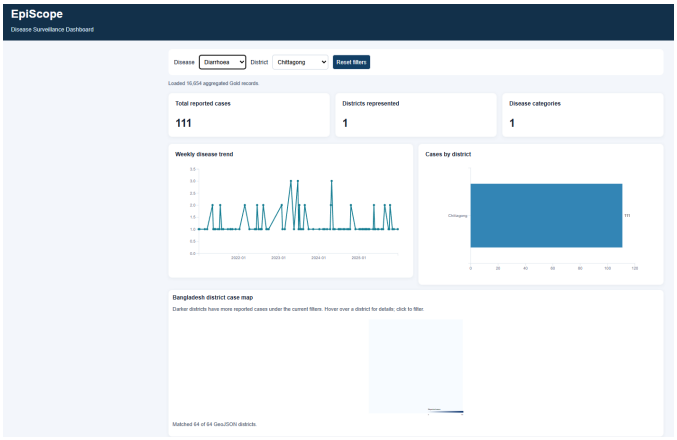


Fig. 2. EpiScope dashboard overview.

can prevent local data files from loading reliably. For example, from the dashboard directory:

```
python -m http.server 8000
```

The dashboard can then be opened at `http://localhost:8000`.

D. Dashboard Verification

The Member 3 report records local verification of aggregated data loading, filters, KPI cards, the weekly trend chart, and the district chart. The local D3.js v7 library was included so the dashboard does not need to retrieve that library from a third-party CDN. JavaScript syntax was checked with Node.js.

Final map rendering and map interaction should be confirmed in the final browser demonstration. The complete coordinated-view requirement, what-if panel, privacy-safe serving controls, and required usability testing with at least two non-CS users still need confirmation.

E. Dashboard Screenshots

The following figures provide visual evidence of the EpiScope dashboard. Each screenshot should be captured from the actual running project and uploaded to the `images` folder in the Overleaf project. The captions below are editable and should be adjusted to describe the specific content visible in each screenshot.

The screenshots are intended to document the implemented interface. They do not, by themselves, establish the correctness of the underlying epidemiological data or the completion of features that remain under verification.

VIII. Testing and Integration

A. Member 1 Pipeline Testing

The documented integration run processed batch:

```
member1_population_v2
```

The results included:

- 100 encounter input records.

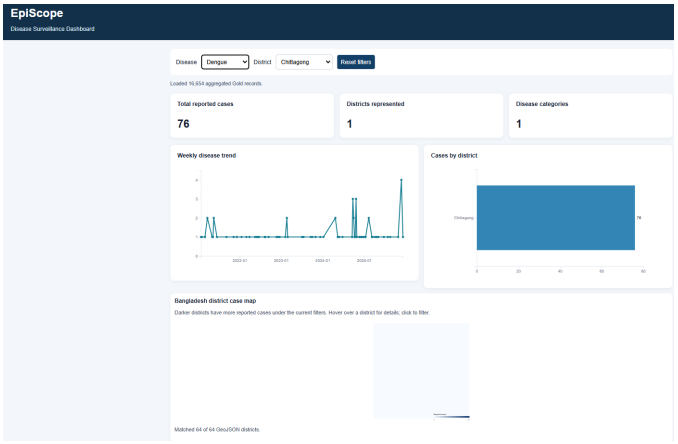


Fig. 3. Dashboard key performance indicators.

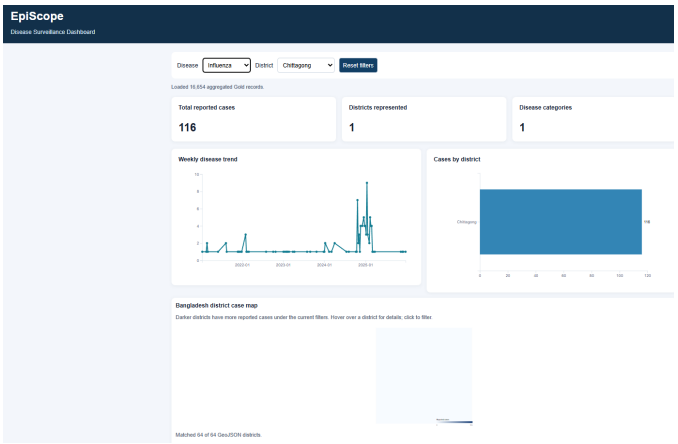


Fig. 4. Weekly disease trend visualization.

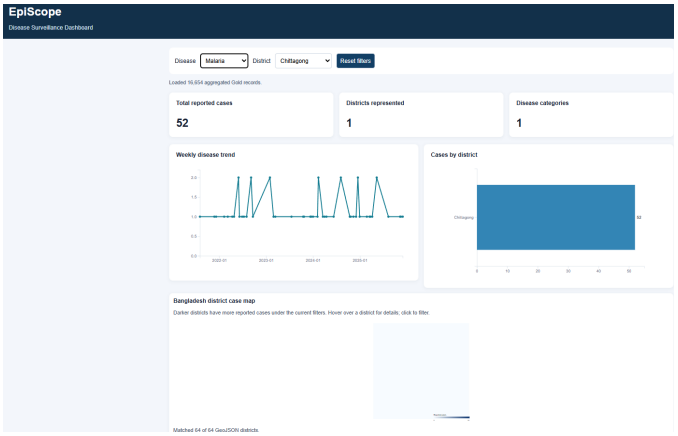


Fig. 5. District-level disease case comparison.

- 98 accepted encounter records.
- 2 quarantined encounter records.
- 3,840 accepted population records.

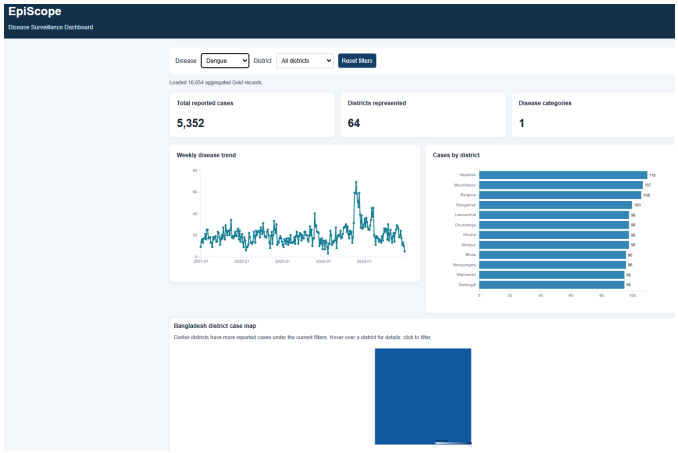


Fig. 6. Geographical disease surveillance visualization.

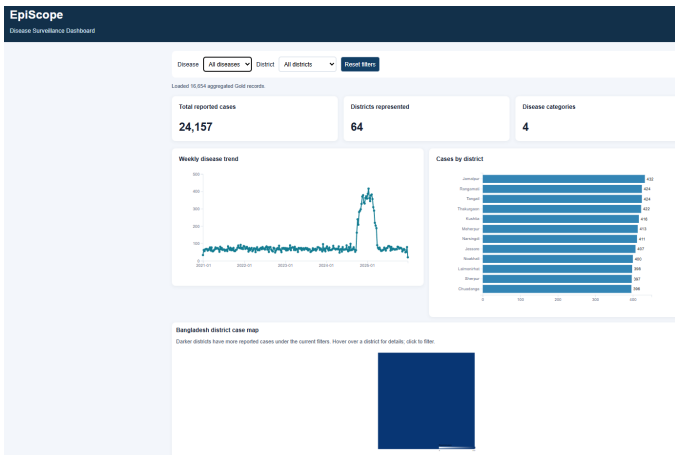


Fig. 7. Dashboard filters and interactive controls.

- 92 weekly district-disease output rows.
- 7,719 weekly facility-reporting output rows.
- 83 affected epidemiological weeks.

The Member 1 report records 27 tests passing. Its source report does not establish that this run is directly comparable to the later Member 4 full-suite run; the results are therefore reported separately.

B. Member 2 Analytics Testing

Member 2 developed five analytics-specific tests covering:

1) Outbreak detection.
2) Forecast calculation.
3) Resource demand estimation.
4) Resource allocation.
5) Rejection of negative resource supply.

All five analytics-specific tests passed.

The Member 2 report records an earlier full-suite result of 32 passed tests and zero failures in approximately 51.92 seconds.

C. Member 4 Testing Activities

Member 4 improved the testing environment and test reliability.

The work included:

- Adding a temporary district-registry fixture for Silver tests.
- Using the generator's existing offline weather fallback.
- Configuring PySpark to use the active Python executable.
- Recording the final test output.
- Improving the project README.

The Silver test module passed four tests, and the generator test module passed three tests.

A Bronze test failed during one full-suite run but passed individually. A subsequent full-suite run also passed. The original intermittent failure was not conclusively explained.

D. Latest Recorded Full Test Result

The latest full test result documented in the Member 4 report was:

TABLE VI
LATEST AUTOMATED TEST RESULTS

Metric	Result
Tests Passed	38
Tests Failed	0
Execution Time	58.79 seconds
Exit Code	0
Overall Result	PASS

The test command was:

```
python -m pytest tests -q
```

The output was saved in:

```
docs/testing/pytest-final.txt
```

Passing tests establish the behaviour of the tested scenarios, not complete compliance with every project requirement.

E. Scalability and Optimization

The supplied reports do not contain measured results for scale factors SF=0.25, SF=0.5, and SF=1.0.

Stage runtimes, memory consumption, hardware details, and before-and-after Spark optimization measurements must be collected before making scalability or performance-improvement claims.

IX. FOUR-MEMBER CONTRIBUTIONS

A. Member 1: Md Imtiaz Uddin – Data Engineer

Member 1 was responsible for developing the data engineering foundation of EpiScope.

The main responsibilities included:

- Source data ingestion.
- Bronze metadata management.
- Silver data validation.
- Quarantine of invalid records.
- Patient identifier hashing.
- Age-band reduction.
- Population data integration.
- Gold weekly aggregation.
- Incremental processing.
- Batch lineage.
- Spark SQL catalog registration.
- Pipeline orchestration.

The primary files were:

```
pipeline/bronze.py
pipeline/silver.py
pipeline/gold.py
pipeline/incremental.py
orchestration/run_pipeline.py
```

A major engineering decision was separating raw, validated, and aggregated data into three processing layers.

This improves traceability and makes the pipeline easier to maintain. However, it also requires careful schema coordination between processing stages.

The documented integration run accepted 98 of 100 encounter records and all 3,840 population records.

Explicit verification of the 21-day late-arrival requirement and historical quality-report retention remains outstanding.

B. Member 2: Sabiha Taslim Jannath – Analytics Engineer

Member 2 was responsible for the analytics component.

The main implementation included:

- Outbreak detection.
- Next-week disease forecasting.
- Resource demand estimation.
- Resource allocation.
- Analytics-specific automated tests.

The implementation was developed in:

```
analytics/analytics.py
tests/test_analytics.py
```

Five analytics-specific tests were created and passed.

The outbreak method uses a rolling historical baseline. Forecasting uses up to four recent observations. Resource demand uses a configurable factor, while allocation prioritizes districts with higher estimated demand.

The contribution was developed on the `sabiha-work` branch. Member 2's report records commit `1c84502`, Pull Request #1, and merge commit `f2f31a7`; the pull request was reported as merged into `main`.

Future improvements include forecast evaluation, threshold calibration, and equity-aware allocation.

C. Member 3: Pritam Nath – Visualization Engineer

Member 3 was responsible for the dashboard and visualization component.

The main implementation included:

- Dashboard interface.
- Disease and district filters.
- Reset functionality.

- KPI cards.
- Weekly disease trend chart.
- District comparison chart.
- Bangladesh district map integration.
- Dashboard-facing data export.
- Silver-layer input compatibility: normalize an input field named `admission` to `admitted` when needed by downstream processing.

The primary files include:

```
dashboard/index.html
dashboard/js/dashboard.js
dashboard/js/d3.v7.min.js
export_dashboard_data.py
```

The implementation uses D3.js v7 and geographic data from GeoJSON and a district-centroid registry. Member 3 also updated `pipeline/silver.py` to normalize the `admission` field to `admitted` when the former appears in an input dataset.

The documented contribution is associated with commit `8908ce0` on the `member3-visualization` branch. The source report states that nine intended files were committed and pushed.

Local checks confirmed aggregated data loading and the core dashboard components. The dashboard was served through a local HTTP server during verification.

Final map rendering and map interaction require browser confirmation. The complete coordinated-view requirement, what-if panel, privacy-safe serving outputs, and usability evidence require further confirmation.

D. Member 4: Nusrat Jahan Ananna – Testing and Integration Engineer

Member 4 was responsible for testing, integration, and project documentation.

The main responsibilities included:

- Improving Silver test fixtures.
- Removing a local weather-file dependency from generator testing.
- Configuring the PySpark environment.
- Verifying automated tests.
- Recording testing evidence.
- Improving the project README.

The testing and documentation work included the following commits:

- `073a93b` – test: improve pipeline tests and document results.
- `8333a3f` – docs: improve project README.

Both commits were documented on the `member4-testing-integration` branch.

The latest recorded full test run passed 38 tests with zero failures in 58.79 seconds.

Further work includes scalability testing, Spark optimization evidence, privacy verification, dashboard acceptance, and final integration review.

E. Contribution Summary

X. Version Control and Collaboration

The main GitHub repository is:

https://github.com/Imtiaz-1132/BDA_Assignment

The documented working branches include:

- Member 2: `sabiha-work`.
- Member 3: `member3-visualization`.
- Member 4: `member4-testing-integration`.

Member 2's report identifies Pull Request #1 as merged into the main branch.

Member 3's documented contribution is associated with commit `8908ce0`.

Member 4's testing and README commits are `073a93b` and `8333a3f`.

A separate `visualization-snapshot` branch was created for screenshots.

XI. Ethical, Societal, and Sustainability Considerations

A. Privacy

Patient-related information requires careful handling.

The project uses salted hashing and age-band reduction in the Silver layer. However, pseudonymous identifiers may remain linkable.

Small-cell suppression, subtraction protection, and role-based access must be implemented and verified before real-world release.

B. Reporting Bias

Uneven facility reporting can make some districts appear to have fewer cases because of incomplete reporting.

Facility-reporting summaries can support investigation, but they do not eliminate reporting bias.

C. False Alerts

A simple outbreak threshold may produce false alerts or miss gradual increases.

Forecasting results should be interpreted as analytical estimates rather than definitive predictions.

D. Resource Equity

The current greedy allocation method prioritizes higher estimated demand but does not guarantee equitable distribution.

Future work should consider minimum service levels, geographic accessibility, resource capacities, and transparent prioritization criteria.

E. Sustainability

The project demonstrates batch processing and Parquet storage.

However, measured storage cost, compute cost, energy consumption, and scalability comparisons were not supplied. Sustainability claims should be supported by actual experiments.

TABLE VII
TEAM CONTRIBUTION SUMMARY

Member	Role	Main Contribution
Md Imtiaz Uddin	Data Engineer	Bronze, Silver, and Gold pipeline, validation, privacy transformations, population processing, incremental processing, lineage, and orchestration.
Sabiha Taslim Jannath	Analytics Engineer	Outbreak detection, forecasting, resource demand estimation, resource allocation, and analytics tests.
Pritam Nath	Visualization Engineer	D3.js dashboard, filters, KPI cards, charts, geographic map integration, and data export.
Nusrat Jahan Ananna	Testing and Integration Engineer	Test fixtures, offline generator testing, PySpark configuration, test evidence, and README improvement.

XII. LIMITATIONS AND FUTURE WORK

The following requirements need further verification:

1) Explicitly verify the required 21-day late-arrival behaviour.
2) Verify historical per-batch Gold quality-report retention.
3) Implement and test suppression of counts from 1 to 4.
4) Protect against subtraction-based disclosure.
5) Verify role-based views.
6) Complete and verify the dashboard's required coordinated views.
7) Confirm the what-if panel.
8) Conduct usability testing with at least two non-CS users.
9) Evaluate outbreak detection using appropriate ground truth and metrics.
10) Evaluate forecasting using MAE, RMSE, or other suitable measures.
11) Compare greedy allocation with a formally defined optimization method.
12) Run scalability experiments using SF=0.25, SF=0.5, and SF=1.0.
13) Document at least two Spark optimizations with measured before-and-after results.
14) Verify all branches, pull requests, and final integration status.
15) Complete a clean end-to-end run and create the final release tag.

XIII. CONCLUSION

EpiScope establishes an academic foundation for disease surveillance analytics.

Its data engineering pipeline ingests heterogeneous datasets, validates and standardizes records, applies privacy-related transformations, and generates weekly district-disease and facility-reporting summaries.

The analytics module implements baseline outbreak detection, next-week forecasting, resource demand estimation, and demand-prioritized allocation.

The D3.js dashboard provides filters, KPI summaries, weekly trends, district comparisons, and geographic map integration.

The four team members contributed to data engineering, analytics, visualization, and testing and integration.

The documented integration run accepted 98 of 100 encounter records and all 3,840 population records. The latest recorded full automated test run passed 38 tests with zero failures in 58.79 seconds.

These results demonstrate the functionality of the tested components, but do not establish complete compliance with every assignment requirement.

Privacy suppression, subtraction protection, role-based access, dashboard completeness, usability evidence, formal analytics evaluation, and scalability measurements remain important future tasks.

EpiScope should therefore be presented as a functioning academic prototype with clearly documented achievements and limitations, rather than as a production-ready clinical system.

REFERENCES

[1] Department of Computer Science and Engineering, Premier University, "CSE 4345 – Big Data Analytics: Complex Engineering Problem Assignment, EpiScope," Spring 2026. [Verify the exact assignment document details.]

[2] Apache Software Foundation, "Apache Spark Documentation." Available: https://spark.apache.org/docs/latest/

[3] M. Bostock, V. Ogievetsky, and J. Heer, "D^3: Data-Driven Documents," IEEE Transactions on Visualization and Computer Graphics, vol. 17, no. 12, pp. 2301–2309, 2011.

[4] World Health Organization, Technical Guidelines for Integrated Disease Surveillance and Response, 3rd ed. [Verify the exact publication used.]

[5] L. Sweeney, "k-anonymity: A model for protecting privacy," International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, vol. 10, no. 5, pp. 557–570, 2002.

[6] pytest Development Team, "pytest Documentation." Available: https://docs.pytest.org/

APPENDIX A
INDIVIDUAL CONTRIBUTION REFLECTIONS

A. *Md Imtiaz Uddin – Data Engineer*

I developed the data engineering foundation of EpiScope, including source ingestion, Bronze metadata, Silver validation and quarantine, patient-identifier hashing, age-band reduction, population processing, Gold aggregation, incremental processing, lineage, and orchestration.

I selected a layered Bronze–Silver–Gold architecture to separate raw, validated, and analytical data. This improves traceability but requires careful schema coordination.

The integration run accepted 98 of 100 encounter records and all 3,840 population records.

Explicit verification of the 21-day late-arrival requirement and historical quality-report retention remains outstanding.

B. Sabiha Taslim Jannath – Analytics Engineer

I implemented baseline outbreak detection, next-week forecasting, resource-demand estimation, and resource allocation.

I also developed five analytics-specific unit tests, all of which passed.

I selected rolling-baseline and moving-average methods because they are transparent starting points.

The allocation method prioritizes higher estimated demand but is a greedy baseline rather than a formal optimization model.

Model evaluation, threshold calibration, and equity-aware allocation remain future work.

C. Pritam Nath – Visualization Engineer

I developed the D3.js dashboard interface, disease and district filters, reset control, KPI cards, weekly trend chart, district comparison chart, and Bangladesh district map integration.

I added `export_dashboard_data.py`, packaged D3.js v7 locally, and updated `pipeline/silver.py` to normalize `admission` to `admitted` when needed by downstream processing.

Local checks confirmed aggregated data loading, filters, KPI cards, and the trend and district charts; JavaScript syntax was checked with Node.js. The dashboard should be served over local HTTP for reliable data loading.

Final map rendering, complete coordinated views, what-if functionality, privacy-safe serving outputs, and usability evidence require further confirmation.

My documented contribution is associated with commit `8908ce0` on the `member3-visualization` branch.

D. Nusrat Jahan Ananna – Testing and Integration Engineer

I improved test reliability by adding a temporary district-registry fixture, using the generator's existing offline weather fallback, and configuring PySpark to use the active Python interpreter.

I recorded the final test output and improved the project README.

The latest recorded full test run passed 38 tests with zero failures in 58.79 seconds.

An earlier intermittent Bronze test failure was not conclusively explained.

Further work includes scalability testing, Spark optimization evidence, privacy verification, dashboard acceptance, and final integration review.

APPENDIX B
TEAM INFORMATION AND CONTRIBUTION AGREEMENT

Complete all placeholders and obtain agreement from all members before submission.

TABLE VIII
TEAM INFORMATION

Field	Information
Group Number	[Insert]
Md Imtiaz Uddin Student ID	[Insert]
Sabiha Taslim Jannath Student ID	[Insert]
Pritam Nath Student ID	[Insert]
Nusrat Jahan Ananna Student ID	[Insert]
Submission Date	[Insert]
Presentation Video URL	[Insert]
Final Release Branch/Tag	[Insert after verification]

TABLE IX
CONTRIBUTION AGREEMENT

Member	Share	Signature
Md Imtiaz Uddin	[%]	[Sign]
Sabiha Taslim Jannath	[%]	[Sign]
Pritam Nath	[%]	[Sign]
Nusrat Jahan Ananna	[%]	[Sign]
Total	100%	

APPENDIX C
FINAL SUBMISSION CHECKLIST

1) Insert group number and all student IDs.
2) Insert the final presentation video URL.
3) Obtain all members' approval of contribution descriptions and percentages.
4) Verify current GitHub branch, pull-request, and merge status.
5) Upload `snap1.png` through `snap6.png` into the Overleaf `images/` folder; verify each caption against its actual image and confirm map rendering separately.
6) Complete or explicitly mark pending the what-if and usability requirements.
7) Test small-cell suppression and subtraction protection.
8) Verify role-based access.
9) Run required scale-factor experiments.
10) Document at least two Spark optimizations with measured evidence.
11) Verify the 21-day late-arrival behaviour.
12) Verify historical quality-report retention.
13) Check references and dataset licences.
14) Compile and check the course page limit.